

FACTORY METHOD PATTERN

Document.java:

```
package Exercise2;

public interface Document{
    void open();
}
```

DocumentFactory.java:

```
package Exercise2;

public abstract class DocumentFactory {
    public abstract Document createDocument();
}
```

ExcelDocument.java:

```
package Exercise2;

public class ExcelDocument implements Document{
    @Override
    public void open(){
        System.out.println("Opening Excel Document!");
    }
}
```

ExcelDocumentFactory.java:

```
package Exercise2;

public class ExcelDocumentFactory extends DocumentFactory{
    @Override
    public Document createDocument(){
        return new ExcelDocument();
    }
}
```

PdfDocument.java:

```
package Exercise2;
```

```
public class PdfDocument implements Document{
    @Override
    public void open(){
        System.out.println("Opening PDF Document!");
    }
}
```

PdfDocumentFactory.java:

```
package Exercise2;

public class PdfDocumentFactory extends DocumentFactory{
    @Override
    public Document createDocument(){
        return new PdfDocument();
    }
}
```

WordDocument.java:

```
package Exercise2;

public class WordDocument implements Document{
    @Override
    public void open(){
        System.out.println("Opening Word Document!");
    }
}
```

WordDocumentFactory.java:

```
package Exercise2;

public class WordDocumentFactory extends DocumentFactory{
    @Override
    public Document createDocument(){
        return new WordDocument();
    }
}
```

FactoryMethodTest.java:

```
package Exercise2;
```

```

public class FactoryMethodTest {
    public static void main(String[] args) {
        DocumentFactory wf= new WordDocumentFactory();
        Document w= wf.createDocument();
        w.open();
        DocumentFactory pf= new PdfDocumentFactory();
        Document p= pf.createDocument();
        p.open();
        DocumentFactory ef= new ExcelDocumentFactory();
        Document e= ef.createDocument();
        e.open();
    }
}

```

